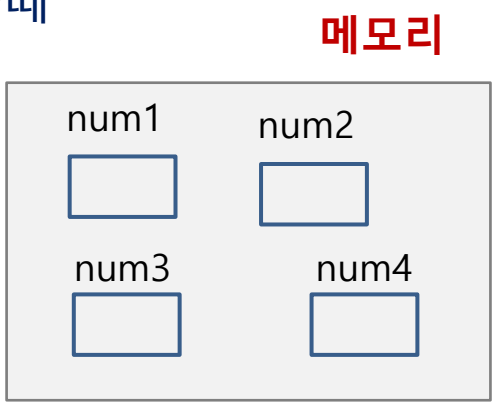
3장. 자료구조(리스트, 딕셔너리, 튜플)

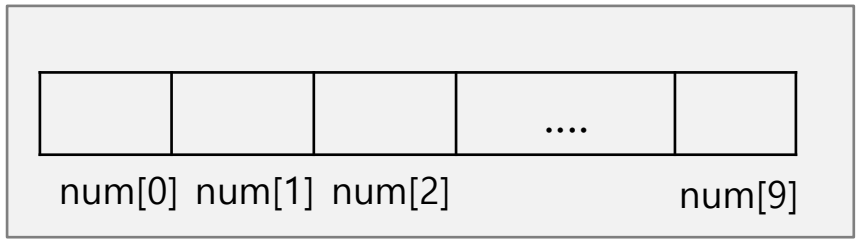
python™

리스트(배열) 사용의 필요성

- **리스트(배열) 사용의 필요성**
  - 정수 10개를 이용한 학생의 성적 프로그램을 만들때 10개의 변수를 선언
    (num1, num2, num3… num10)
  - 정보가 흩어진 채 저장되고, 변수 이름이 많아 비효율적이고 관리하기 어렵다.



- **리스트(배열) 사용의 장점**
  - 인덱스를 이용하여 순차(순서)적으로 관리할 수 있다 -> 효율적이다.



리스트(list)란?

- **리스트(list)란?**

  - 여러 개의 연속적인 값을 저장하고자 할 때 사용하는 자료형이다.
    (변수는 1개의 값만을 저장하고 변경할 수 있다.)
  - 리스트의 맨 앞요소의 위치(인덱스)는 0번이다.

- **리스트의 생성**

리스트 이름 = [요소1, 요소2, 요소3...]

season = ["봄", "여름", "가을", "겨울"]

number = [1. 2, 3, 4, 5]

리스트(list)란?

- **챗GPT 프롬프트 입력**

+ 파이썬 리스트 자료구조에 대해 설명해줘

1. 리스트란?

리스트는 여러 개의 값을 순서대로 저장하는 자료구조입니다.
대괄호 `[]` 를 사용해서 만들어요.

python 코드 복사

```python
numbers = [1, 2, 3, 4]
fruits = ["apple", "banana", "cherry"]
mixed = [1, "hello", 3.14, True]
```

👉 서로 **다른 자료형을 섞어서** 저장할 수 있다는 게 특징이에요.

리스트(list)란?

2. 리스트의 특징

✅ 순서가 있다 (Ordered)

- 인덱스(index)로 접근 가능
- 인덱스는 **0부터 시작**

```python
fruits = ["apple", "banana", "cherry"]
print(fruits[0])   # apple
print(fruits[-1])  # cherry
```

✅ 변경 가능하다 (Mutable)

리스트 안의 값은 **수정, 추가, 삭제**가 가능해요.

```python
fruits[1] = "orange"
print(fruits)  # ['apple', 'orange', 'cherry']
```

✅ 중복 허용

```python
nums = [1, 2, 2, 3]
```

리스트(list)의 활용

- 정수형 리스트

```
my_list = [10, 20, 30, 40]
print(my_list) #[10, 20, 30, 40]
print(type(my_list)) #<class 'list'>

# 인덱스(위치)로 요소에 접근
print(my_list[0]) #10
print(my_list[1]) #20
print(my_list[2]) #30
print(my_list[3]) #40
print(my_list[-1]) #40
print(my_list[-2]) #30

# 반복문으로 전체 요소 조회
for item in my_list:
    print(item)
```

리스트(list)의 활용

- 정수형 리스트

```
# 요소 변경(수정)
my_list[1] = 50
print(my_list)

# 요소 추가 - 맨 뒤에 추가됨 - append(요소)
my_list.append(100)
print(my_list)

# 요소 삭제 - del 명령어
# del my_list[2] #30 삭제
my_list.remove(30) #값으로 삭제
print(my_list)
```

리스트(list)의 생성

- 문자형 리스트

```
carts = ["라면", "달걀", "커피", "토마토"]
print(carts)
print(type(carts)) #<class 'list'>

# 리스트의 길이(크기)
print(len(carts)) #4

# 요소 조회
print(carts[2]) #"커피"
print(carts[-1]) #"토마토"

# 범위 - 여러개 조회(슬라이싱)
# 종료값-1 인덱스로 조회
print(carts[0:2]) #['라면', '달걀']
print(carts[1:4]) #['달걀', '커피', '토마토']
print(carts[1:]) #['달걀', '커피', '토마토']
print(carts[:]) #['라면', '달걀', '커피', '토마토']
```

리스트(list)의 생성

- **문자형 리스트**

```
# 요소 변경
carts[2] = "콜라"
print(carts)

# 요소 추가 - 맨 뒤에 추가
carts.append("감자")
print(carts) #['라면', '달걀', '콜라', '토마토', '감자']

# 요소 삭제 - remove(요소) /pop()
carts.remove("달걀")
print(carts)

last_item = carts.pop() # 맨 뒤에서 삭제
print("삭제된 아이템:", last_item) #감자
print(carts) #['라면', '콜라', '토마토']
```

리스트(list) 반복 – in 사용

- for 변수 in 리스트

```
my_list = [10, 20, 30, 40, 50]
print(20 in my_list)  # True
print(25 in my_list)  # False

print("for문을 사용한 리스트 출력:")

for item in my_list:
    print(item, end=' ')
print()

# 40보다 큰 요소만 출력
print("40보다 큰 요소 출력:")

for item in my_list:
    if item > 40:
        print(item, end=' ')
print()
```

리스트(list)의 연산

- 리스트의 연산 – 합계, 평균, 최대값, 최소값 구하기

```
lst = [10, 20, 30, 40]

# 요소의 개수
print(len(lst)) #4

# 요소의 합계
total = lst[0] + lst[1] + lst[2] + lst[3]
print(total) #100

# 반복문으로 요소의 합계 구하기
total = 0  # 합계 초기화(필수)
for item in lst:
    # print(item)
    # total = total + item
    total += item
print("요소의 합계:", total)
```

```
"""
    1회전 item -> 10
    2회전 item -> 20
    3회전 item -> 30
    4회전 item -> 40
"""
```

리스트(list)의 연산

- 리스트의 연산 – 합계, 평균, 최대값, 최소값 구하기

```
# 평균 = 합계 / 개수
average = total / len(lst)
print("요소의 평균:", average) #25.0

# 최대값
max_value = lst[0]  # 최대값 초기화(필수)
for item in lst:
    if item > max_value:
        max_value = item
print("요소의 최대값:", max_value) #40

# 최소값
min_value = lst[0]  # 최소값 초기화(필수)
for item in lst:
    if item < min_value:
        min_value = item
print("요소의 최소값:", min_value) #10
```

리스트(list) 반복 – in 사용

- **if 변수 in [list]**

  리스트 내부에 값이 있으면 True, 없으면 False

```
# 음식 분류하기 - 한식, 일식, 중식
foods = ["비빔밥", "짜장면", "초밥", "김치찌게"]

for food in foods:
    if food in ["짜장면",  "짬뽕"]:
        print(f'{food}는(은) 중식입니다.')
    elif food in ["초밥", "우동"]:
        print(f'{food}는(은) 일식입니다.')
    else:
        print(f'{food}는(은) 한식입니다.')
```

```
비빔밥는(은) 한식입니다.
짜장면는(은) 중식입니다.
초밥는(은) 일식입니다.
김치찌게는(은) 한식입니다.
```

리스트(list)의 주요 함수

- 리스트의 주요 메서드(함수)

함수	기능	사용 예
append()	요소 추가	a = [1, 2, 3] a.append(4) a = [1, 2, 3, 4]
insert()	특정 위치에 추가	a = [2, 4, 5] a.insert(1,3) #1번 위치에 3 삽입 a = [2, 3, 4, 5]

pop()	요소 삭제	a = [1, 2, 3, 4, 5]
		a.pop() # 마지막 위치의 요소 제거
		a = [1, 2, 3, 4]
		a.pop(1) #1 위치의 2 제거
		a = [1, 3, 4]

remove()	특정 요소 삭제	s = ['모닝', 'BMW', 'BENZ', '스포티지']
		s.remove('BMW') #요소 직접 삭제
		s = ['모닝', 'BENZ', '스포티지']

리스트(list)의 주요 함수

- 리스트의 주요 메서드(함수)

함수	기능	사용 예
sort()	정렬	a = [1, 4, 2, 3] a.sort() [1, 2, 3, 4]
reverse()	뒤집기	lower = ['b', 'c', 'a'] lower.reverse() ['a', 'b', 'c']
extend(리스트)	리스트의 끝에 리스트 추가	li = ['a', 'b'], li.extend(['c','d']) ['a', 'b', 'c', 'd'],
copy()	리스트 복사	n = [1, 2, 3] m = n.copy()

리스트(list)의 주요 함수

- 리스트의 주요 메서드(함수)

```
# append() - 요소 추가
a1 = [1, 2, 3, 4, 5]
a2 = []  #빈 리스트 생성

# a1의 요소를 a2에 저장함
for item in a1:
    a2.append(item)
print("a2 =", a2)

# a3 리스트에 a1 요소중 홀수만 저장
a3 = []
for item in a1:
    if item % 2 == 1:
        a3.append(item)
print("a3 =", a3) #a3 = [1, 3, 5]
```

리스트(list) 내포

- 리스트 내포 사용하기

[표현식 for 항목(요소) in 리스트]

```
a4 = [item for item in a1]
print("a4 =", a4) #a4 = [1, 2, 3, 4, 5]

a5 = [item for item in a1 if item % 2 == 1]
print("a5 =", a5) #a5 = [1, 3, 5]
```

리스트(list)의 주요 함수

- 리스트의 주요 메서드(함수)

```
numbers = [5, 2, 9, 1, 5]

# 요소 정렬 - sort()
numbers.sort() #오름차순
print("sort 후:", numbers) #[1, 2, 5, 5, 9]

# 요소 뒤집기 - reverse()
numbers.reverse()
print("reverse 후:", numbers) #[9, 5, 5, 2, 1]

# 리스트 복사 - copy()
copied_numbers = numbers.copy()
print("copy 후:", copied_numbers) #[9, 5, 5, 2, 1]

# 여러개의 요소 추가 - extend(리스트)
# copied_numbers.append(11, 12)  #오류발생
copied_numbers.extend([11, 12])
print("10, 11 추가후:", copied_numbers) # [9, 5, 5, 2, 1, 10, 11, 12]
```

문자열 인덱싱, 슬라이싱

◆ **문자열은 특별한 1차원 리스트이다.**

문자열(시작번호:끝번호)

※ 끝번호는 (끝번호 -1)과 같다

```
# 문자를 저장한 리스트
my_list = ['h', 'e', 'l', 'l', 'o']
print(my_list)
print(my_list[0]) # h
print(my_list[-1]) # 0
print(my_list[0:-1]) # ['h', 'e', 'l', 'l']
print(len(my_list)) #5

# 문자열 - 특별한 1차원 리스트
greeting = "hello"
print(greeting[0]) # h
print(greeting[-1]) # 0
print(greeting[0:-1]) # hell
print(len(greeting)) #5
```

문자열 함수

- **문자열 함수(메서드) 정리**

메서드	설명
split()	s = 'banana, grape, kiwi' s = fruit.split(',') [구분기호로 나누고 리스트로 만듬] s ['banana', ' grape', ' kiwi']
replace()	s = 'Hello, World' s = s.replace('World', 'Korea') [문자를 변경함] 'Hello, Korea'
find()	s = "Hello" s.find('H') 0 s.find('k') -1 [문자열이 존재하는 위치 반환. 없으면 -1반환]
strip()	s = " Hi, lee" s.strip() Hi, lee

문자열 함수

- 문자열 함수(메서드)

```
# 대소문자 변환
print("소문자 변환:", greeting.lower()) #hello
print("대문자 변환:", greeting.upper()) #HELLO

# 문자열 변경 - replace(변경전 문자, 변경후 문자)
s1 = "Hello, World"
print(s1.replace('World', 'Korea')) #Hello, Korea

# 특정 문자의 시작 위치(인덱스) 찾기 - find(문자)
print(s1)
print("문자열에서 'World'의 위치:", s1.find('World')) #7
# 찾는 문자가 문자열에 없으면 -1을 반환
print("문자열에서 'World'의 위치:", s1.find('Seoul')) #-1
```

문자열 함수

- 문자열 함수(메서드)

```
# split() -> 문자열을 구분기호로 분리하는 함수
csv = "apple,banana,strawberry"
fruits = csv.split(',') #문자열이 리스트형태로 변환됨
print(fruits) #['apple', 'banana', 'strawberry']
print(fruits[0]) #apple
print(fruits[-1]) #strawberry

# strip() -> 문자열의 양쪽 공백 제거
s2 = "   Hello, World!   "
print(s2.strip()) #Hello, World!
print(s2.lstrip()) #Hello, World!, 왼쪽 공백 제거
print(s2.rstrip()) #   Hello, World!, 오른쪽 공백 제거
```

실습 문제 – 문자열 처리

2025년 민생회복 지원금 신청 프로그램을 아래와 같이 구현하세요.

출생연도 끝자리	접종 요일
1 또는 6	월요일
2 또는 7	화요일
3 또는 8	수요일
4 또는 9	목요일
5 또는 0	금요일

☞ 실행 결과

```
출생연도 입력: 2023
신청일은 수요일입니다.
```

```
출생연도 입력: 2010
출생연도는 1900년부터 2006년 사이여야 합니다.
```

2차원 리스트(list)

- **2차원 리스트의 선언 및 생성**
  - 리스트 내부에 리스트를 가진 자료 구조이다.
  - 행과 열의 표(테이블) 형태를 이루고 있다.

리스트 이름 = [요소1, 요소2, **[요소1, 요소2, 요소3]**]

	열1	열2
행1	a[0][0]	a[0][1]
행2	a[1][0]	a[1][1]
행3	a[2][0]	a[2][1]

2차원 리스트(list)

- 2차원 리스트 생성 및 출력

```
matrix = [
    [1, 2, 3],  #1행
    [4, 5, 6]   #2행
]

# 출력
print(matrix) #[[1, 2, 3], [4, 5, 6]]
print(type(matrix)) #<class 'list'>

# 특정 행 출력
print(matrix[0]) #[1, 2, 3]
print(matrix[1]) #[4, 5, 6]

# 특정 값 출력
print(matrix[0][0]) # 0행 0열, 1
print(matrix[0][1]) # 0행 1열, 2
print(matrix[1][2]) # 1행 2열, 6
```

2차원 리스트(list)

- **2차원 리스트 생성 및 출력**

```
# 전체 값 출력
for row in matrix:  #행
    for val in row: #열 - 0행 [1, 2, 3], 1행 [4, 5, 6]
        print(val, end=" ") #1 2 3 4 5 6
    print() #행이 끝날 때 줄바꿈

# 요소 변경
matrix[0][1] = 20

# 요소 추가
matrix.append([7, 8, 9]) #3행에 [7, 8, 9] 추가
print(matrix) #[[1, 20, 3], [4, 5, 6], [7, 8, 9]]

# 요소 삭제
del matrix[1] #2행 삭제
print(matrix) #[[1, 20, 3], [7, 8, 9]]
```

2차원 리스트(list)

- 2차원 리스트의 연산

```
d = [
    [10, 20],
    [30, 40],
    [50, 60, 70]
]

total = 0  # 총합 변수
count = 0  # 요소 개수 변수

for row in d:
    for value in row:
        total += value
        count += 1

# 평균 계산
average = total / count

print("총합:", total)  # 총합: 280
print("요소 개수:", count) # 요소 개수: 7
print("평균:", average) # 평균: 40.0
```

2차원 리스트(list)

- **성적 통계**

```
scores = [
    ["철수", 80, 90, 85],
    ["영희", 70, 88, 92],
    ["민수", 95, 85, 90]
]

for student in scores:
    name = student[0]
    total = sum(student[1:])
    avg = total / len(student[1:])
    print(f"{name}: 총점 {total}, 평균 점수: {avg:.2f}")
```

```
철수: 총점 255, 평균 점수: 85.00
영희: 총점 250, 평균 점수: 83.33
민수: 총점 270, 평균 점수: 90.00
```

딕셔너리(Dictionary)

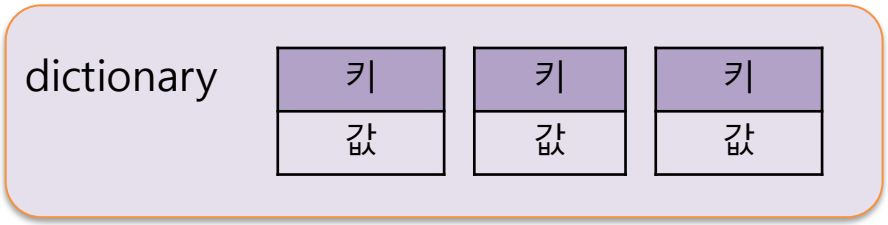
❖ **딕셔너리**

리스트 처럼 여러 개의 값을 저장할 수 있고, 키(key)와 값(value)으로 대응시켜 저장하는 자료구조이다.

중괄호{ }를 사용한다.

딕셔너리 이름 = **{ 키:값, 키:값....}**

{ 'name': '한국민', 'age': 28 }



딕셔너리(Dictionary)

❖ 딕셔너리 주요 메서드

함수	사용 예
d[key] = value	d = {'Tomas':13, 'Jane':9} d['Mike'] = 10 **# 요소 추가** {'Tomas':13, 'Jane':9, 'Mike':10 }
del d[key]	del d['Jane'] **#요소 삭제** {'Tomas':13, 'Mike':10 }
d.pop(key)	d.pop('Mike') 10 {'Tomas':13}
clear()	d.clear() # d={ } 빈 딕셔너리
d.keys()	d.keys() **# 모든 키 가져오기** d_keys(['Tomas', 'Mike'])
d.values()	d.Values() **# 모든 값 가져오기** d_values([13, 10])
d.get(key)	d.get('Jane') #9

딕셔너리(Dictionary)

❖ 딕셔너리 생성 및 관리

```
student = {
  "name": "한강",
  "age": 21,
  "university": "한강대학교"
}

print(student) #{'name': '한강', 'age': 21, 'university': '한강대학교'}
print(type(student)) #<class 'dict'>

# 요소에 접근(검색)
# 1. 방법1
print(student["name"]) #한강
print(student["age"]) #21
print(student["university"]) #한강대학교

# 2. 방법2 - get(key)
print(student.get("name"))
print(student.get("age"))
print(student.get("university"))
```

딕셔너리(Dictionary)

❖ 딕셔너리 생성 및 관리

```
# 요소 수정
student["age"] = 31
print(student)

# 요소 삭제 - pop(key)
student.pop("university")
print("요소 삭제후 딕셔너리:", student)

print(list(student.keys())) #['name', 'age', 'major']
print(list(student.values())) #['한강', 31, '전자공학과']

# 키, 값 목록
keys = student.keys()
values = student.values()
print("키 목록:", keys) #키 목록: dict_keys(['name', 'age', 'major'])
print("값 목록:", values) #값 목록: dict_values(['한강', 31, '전자공학과'])

for key in student.keys():
    print(key, ":", student[key])
```

딕셔너리(Dictionary)

- **용어 사전 만들기**

```
♠ 컴퓨터 용어 사전 ♠
검색할 용어를 입력하세요 (종료: q or Q): 변수
변수: 데이터를 저장하기 위해 이름을 붙인 공간
검색할 용어를 입력하세요 (종료: q or Q): 함수
'함수'은(는) 사전에 없습니다.
검색할 용어를 입력하세요 (종료: q or Q): q
프로그램을 종료합니다.
```

1. dictionary 자료구조에 컴퓨터 용어와 정의를 저장한다.
2. 용어를 계속 반복해서 검색 할 수 있다.
3. 검색한 용어가 없으면 정의된 단어가 없음을 알려준다.
4. 검색을 종료하려면 ‘q’ 또는 ‘Q’를 입력한다.

딕셔너리(Dictionary)

- 용어 사전 만들기

```
print("♠ 컴퓨터 용어 사전 ♠")

dic = {
    "이진수": "0과 1로 이루어진 수 체계",
    "변수": "데이터를 저장하기 위해 이름을 붙인 공간",
    "RAM": "Random Access Memory - 임의 접근 메모리",
    "CPU": "Central Processing Unit - 중앙 처리 장치"
}

while True:
    word = input("검색할 용어를 입력하세요 (종료: q or Q): ")
    if word.lower() == 'q':
        print("프로그램을 종료합니다.")
        break
    elif word in dic:
        # print(f"{word}: {dic[word]}")
        print(f"{word}: {dic.get(word)}")
    else:
        print(f"'{word}'은(는) 사전에 없습니다.")
```

학생의 성적 관리

- **학생 성적표 프로그램**

```
학생 성적표
이름     국어     영어     수학     총점     평균
이대한   95       80       80       255      85.00
박민국   80       75       75       230      76.67
오상식   90       85       90       265      88.33
```

1. dictionary 자료구조 사용하여 학생 리스트를 생성
2. 학생 성적표를 출력
3. 총점과 평균을 계산

학생의 성적 관리

- 학생 성적표 프로그램

```
student_list = [
    {"name": "이대한", "kor": 95, "eng": 80, "math": 80},
    {"name": "박민국", "kor": 80, "eng": 75, "math": 75},
    {"name": "오상식", "kor": 90, "eng": 85, "math": 90}
]

# 학생 리스트 출력
print("첫 번째 요소 검색:", student_list[0])
print("첫 번째 학생의 이름:", student_list[0]["name"])

print("학생 성적표")
print("이름\t국어\t영어\t수학")
for student in student_list:
    name = student["name"]
    kor = student["kor"]
    eng = student["eng"]
    math = student["math"]
    print(f"{name}\t{kor}\t{eng}\t{math}")
```

실습 문제 – 딕셔너리

다음의 실행 결과가 나오도록 빈 칸을 작성하시오.(파일: member.py)

```
member = {"이름": "신유빈", "나이": 20, "특기": "탁구"}
result = [                    ]

print(member)
print(result)
```

☞ 실행 결과

```
{'이름': '신유빈', '특기': '탁구'}
20
```

튜플(tuple)

- **튜플(tuple)**
  - 튜플의 요소를 변경(추가, 수정, 삭제)할 수 없다.
  - 요소 추가는 초기화나 튜플간 합치기를 하면 가능함
  - 리스트처럼 동일한 방식으로 인덱싱과 슬라이싱 가능함
  - 소괄호( ) 를 사용한다.

튜플 이름 = **(요소1, 요소2....)**

```
t1 = ()
t2 = (1, )
t3 = (1, 2, 3)
t4 = ('a', 'b', 'c')
```

튜플(tuple)

- **튜플 자료형**

```python
# 튜플 생성
t = (1, 2, 3)
print(t)  # (1, 2, 3)
print(type(t))  # <class 'tuple'>

# 특정 요소 접근
print("첫 번째 요소:", t[0])  # 1
print("두 번째 요소:", t[1])  # 2
print("마지막 요소:", t[-1])  # 3

# 튜플 길이
print("튜플의 길이:", len(t))  # 3

# 슬라이싱
print("첫 두 요소:", t[0:2])  # (1, 2)
print("두 번째 요소부터 끝까지:", t[1:])  # (2, 3)
```

```
(1, 2, 3)
<class 'tuple'>
첫 번째 요소: 1
두 번째 요소: 2
<class 'tuple'>
첫 번째 요소: 1
두 번째 요소: 2
첫 번째 요소: 1
두 번째 요소: 2
두 번째 요소: 2
마지막 요소: 3
마지막 요소: 3
튜플의 길이: 3
첫 두 요소: (1, 2)
두 번째 요소부터 끝까지: (2, 3)
단일 요소 튜플: (5,)
튜플 연결: (1, 2, 3, 5)
```

튜플(tuple)

- **튜플 자료형**

```
# 튜플은 변경 불가능(immutable)
# t[0] = 10  # 오류 발생

# 튜플 삭제 불가능
# del t[1]  # 오류 발생

#  단일 요소 튜플 생성
t2 = (5)
print("단일 요소 튜플 아님:", t2)  # 5

t2 = (5,)
print("단일 요소 튜플:", t2)  # (5,)

# 튜플 연결
t3 = t + t2
print("튜플 연결:", t3)  # (1, 2, 3, 5)
```

튜플의 요소는 수정 및 삭제 할 수 없다.

튜플(tuple)

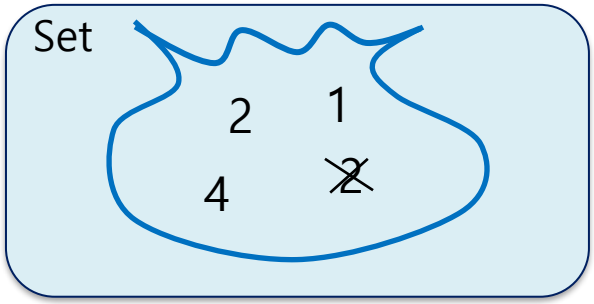
- 튜플 사용 예제

```
# 점수 입력
scores = []
scores.append((80,))
scores.append((70,))
scores.append((90,))
print(scores)

# 과목과 점수 입력
subjects = []
subjects.append(("국어", 90))
subjects.append(("수학", 80))
print(subjects) # [('국어', 90), ('수학', 80)]

# 요소 조회
print(subjects[0]) # ('국어', 90)
print(subjects[1]) # ('수학', 80)

# 이차원 검색
print(subjects[0][1]) # 90
print(subjects[1][1]) # 80
```

집합 자료형(set)

- 집합 자료형
  - 집합에 관련된 것을 쉽게 처리하기 위해 만든 자료구조
  - 중복을 허용하지 않고 , 순서가 없다.
  - 중괄호 { }를 사용한다.

집합 이름 = { 요소1, 요소2, 요소3 }



집합(set) 관련 메서드

- set 관련 메서드

함수	사용 예
add(x)	s = {1, 2, 3} s.add(4) # 요소 추가 {4, 1, 2, 3,} # 순서없이 저장
remove(x)	s1 = {1, 3, 4} s1.remove(3) #요소 삭제 {1, 2}

clear()	s1 = {1, 3, 4}
	s1.clear() # 모두 지우기
	set()

x in s	fruits = {"apple", "banana", "grape"} "apple" in fruits True "grape" not in fruits #False

집합 자료형(set)

- 집합 자료형 예제

```
s = {10, 20, 30}
print(s) #{10, 20, 30}
print(type(s)) #<class 'set'>

# 요소 추가
s.add(40)
print(s) #{40, 10, 20, 30}

s.add(100) #새로운 요소 추가
print(s) #{40, 10, 100, 20, 30}

s.add(20) #중복된 요소 추가(변경없음)
print(s) #{40, 10, 100, 20, 30}

# 요소 삭제
s.remove(20) #20 요소 삭제
print(s) #{40, 10, 100, 30}
```

집합 자료형(set)

- **집합 연산자**

연산자	집합
&	교집합
|	합집합
a-b	차집합

```
# 집합 연산자
a = {1, 2, 3}
b = {2, 3, 4}

c = a & b  #교집합
d = a | b  #합집합
e = b - a  #차집합

print(c) #{2, 3}
print(d) #{1, 2, 3, 4}
print(e) #{4}
```

집합 자료형(set)

- 집합 자료형 예제

```
# 집합 연산
s1 = {1, 3, 4}
s2 = {3, 4, 5, 6}

# 합집합
print(s1 | s2) #{1, 3, 4, 5, 6}

# 교집합
print(s1 & s2) #{3, 4}

# 차집합
print(s1 - s2) #{1}
```

집합 자료형(set)

- **집합 자료형 예제**

```
# 빈 집합 생성
s3 = set()
print(s3)  # set()

# 요소 추가
s3.add(10)
s3.add(20)
s3.add(10)  # 중복 추가 시도
print(s3)  # {10, 20}

# 리스트에서 집합으로 변환 (중복 제거)
lst = [1, 2, 2, 3, 4, 4]
unique_set = set(lst)
print(unique_set)  # {1, 2, 3, 4}
```